



Receive Stripe events in your webhook endpoint

Listen for events from Stripe on your webhook endpoint so your integration can automatically trigger reactions.

[Copy for LLM](#)[View as Markdown](#)

Send events to your AWS account

You can send events directly to [Amazon EventBridge](#) as an event destination.

Create an event destination to receive events at an HTTPS webhook endpoint. After you register a webhook endpoint, Stripe can push real-time event data to your application's webhook endpoint when **events** happen in your Stripe account. Stripe uses HTTPS to send webhook events to your app as a JSON payload that includes an **Event object**.

Receiving webhook events helps you respond to asynchronous events, such as when a customer's bank confirms a payment, a customer disputes a charge, or a recurring payment succeeds.

Get started

To start receiving webhook events in your app:

- 1 Create a webhook endpoint handler to receive event data POST requests.
- 2 Test your webhook endpoint handler locally using the Stripe CLI.
- 3 Create a new **event destination** for your webhook endpoint.
- 4 Secure your webhook endpoint.

You can register and create one endpoint to handle several different event types at the same time, or set up individual endpoints for specific events.

Unsupported event type behaviors for organization event destinations

Stripe sends most event types asynchronously, but waits for a response for some event types. In these cases, Stripe behaves differently based on whether or not the event destination responds.

If your event destination receives **Organization** events, those requiring a response have the following limitations:



You can't subscribe to `issuing_authorization.request` for organization destinations.

Instead, set up a **webhook endpoint** in a Stripe account within the organization to subscribe to this event type. Use `issuing_authorization.request` to authorize purchase requests in real-time.

Organization destinations receiving `checkout_sessions.completed` can't **handle redirect behavior** when you embed **Checkout** directly in your website or redirect customers to a Stripe-hosted payment page. To influence Checkout redirect behavior, process this event type with a **webhook endpoint** configured in a Stripe account within the organization.

Organization destinations responding unsuccessfully to an `invoice.created` event can't influence **automatic invoice finalization when using automatic collection**. You must process this event type with a **webhook endpoint** configured in a Stripe account within the organization to trigger automatic invoice finalization.

1 Create a handler



Set up an HTTP or HTTPS endpoint function that can accept webhook requests with a POST method. If you're still developing your endpoint function on your local machine, it can use HTTP. After it's publicly accessible, your webhook endpoint function must use HTTPS.

Set up your endpoint function so that it:

- Handles POST requests with a JSON payload consisting of an **event object** .

- For **organization** event handlers, it inspects the `context` value to determine which account in an organization generated the event, then sets the `Stripe-Context` header corresponding to the `context` value.

- Quickly returns a successful status code (`2xx`) prior to any complex logic that might cause a timeout. For example, you must return a `200` response before updating a customer's invoice as paid in your accounting system.

Note

Use our [interactive webhook endpoint builder](#) to build a webhook endpoint function in your programming language.

Use the Stripe API reference to identify the [thin event objects](#) or [snapshot event objects](#) your webhook handler needs to process.



Example endpoint

This code snippet is a webhook function configured to check for received events from a Stripe account, handle the events, and return a `200` responses. Reference the [snapshot](#) event handler when you use API v1 resources, and reference the [thin](#) event handler when you use API v2 resources.

[Snapshot event handler](#)

[Thin event handler \(Clover+\)](#)

[Thin event handler \(Acacia or Basil\)](#)

When you create a snapshot event handler, use the API object definition at the time of the event for your logic by accessing the event's `data.object` fields. You can also retrieve the API resource from the Stripe API to access the latest and up-to-date object definition.

Ruby



```

1  require 'json'
2
3  # Replace this endpoint secret with your unique endpoint secret key
4  # If you're testing with the CLI, run 'stripe listen' to find the secret key
5  # If you defined your endpoint using the API or the Dashboard, check your webhook secret key
6  endpoint_secret = 'whsec_...';
7
8  # Using Sinatra
9  post '/webhook' do
10     payload = request.body.read
11     event = nil
12
13     begin
14         event = Stripe::Event.construct_from(
15             JSON.parse(payload, symbolize_names: true)
16         )
17     rescue JSON::ParserError => e
18         # Invalid payload
19         status 400
20         return
21     end
22
23     # Check that you have configured webhook signing
24     if endpoint_secret
25         # Retrieve the event by verifying the signature using the raw body and the endpoint
26         signature = request.env['HTTP_STRIPE_SIGNATURE'];
27         begin

```

Using context

Snapshot events

Thin event handler (Clover+)

Thin event handler (Acacia or Basil)

This code snippet is a webhook function configured to check for received events, detect the originating account if applicable, handle the event, and return a `200` response.

Ruby



```
1  require 'json'
2
3  # Using Sinatra
4  post '/webhook' do
5    payload = request.body.read
6    event = nil
7
8    begin
9      event = Stripe::Event.construct_from(
10        JSON.parse(payload, symbolize_names: true)
11      )
12    rescue JSON::ParserError => e
13      # Invalid payload
14      status 400
15      return
16    end
17
18    # Extract the context
19    context = event.context
20
21    # Define your API key variables (ideally loaded securely)
22    ACCOUNT_123_API_KEY = "sk_test_123"
23    ACCOUNT_456_API_KEY = "sk_test_456"
24
25    account_api_keys = {
26      "account_123" => ACCOUNT_123_API_KEY,
27      "account_456" => ACCOUNT_456_API_KEY
28    }
29
30    api_key = account_api_keys[context]
31
32    if api_key.nil?
33      puts "No API key found for context: #{context}"
34      status 400
35      return
36    end
37
38    # Handle the event
39    case event.type
40    when 'customer.created'
41      customer = event.data.object
42
43      begin
44        latest_customer = Stripe::Customer.retrieve(
45          customer.id,
46          { api_key: api_key }
47        )
48        handle_customer_created(latest_customer, context)
49      rescue => e
50        puts "Error retrieving customer: #{e.message}"
51        status 500
```



```

52     return
53   end
54
55   when 'payment_method.attached'
56     payment_method = event.data.object
57
58     begin
59       latest_payment_method = Stripe::PaymentMethod.retrieve(
60         payment_method.id,
61         { api_key: api_key }
62       )
63       handle_payment_method_attached(latest_payment_method, context)
64     rescue => e
65       puts "Error retrieving payment method: #{e.message}"
66       status 500

```



2 Test your handler



Before you go-live with your webhook endpoint function, we recommend that you test your application integration. You can do so by configuring a local listener to send events to your local machine, and sending test events. You need to use the **CLI** to test.

Forward events to a local endpoint

To forward events to your local endpoint, run the following command with the **CLI** to set up a local listener. The `--forward-to` flag sends all **Stripe events** in a **sandbox** to your local webhook endpoint. Use the appropriate CLI commands below depending on whether you use **thin** or snapshot events.

Forward snapshot events

Forward thin events

Use the following command to forward **snapshot events** to your local listener.

Command Line



```
$ stripe listen --forward-to localhost:4242/webhook
```

Note

You can also run `stripe listen` to see events in **Stripe Shell**, although you won't be able to forward events from the shell to your local endpoint.

Useful configurations to help you test with your local listener include the following:

To disable HTTPS certificate verification, use the `--skip-verify` optional flag.

To forward only specific events, use the `--events` optional flag and pass in a comma separated list of events.



Forward target snapshot events

Forward target thin events

Use the following command to forward target snapshot events to your local listener.

Command Line



```
$ stripe listen --events payment_intent.created,customer.created,payment_intent.succeeded  
> --forward-to localhost:4242/webhook
```

To forward events to your local webhook endpoint from the public webhook endpoint that you already registered on Stripe, use the `--load-from-webhooks-api` optional flag. It loads your registered endpoint, parses the path and its registered events, then appends the path to your local webhook endpoint in the `--forward-to path`.

Forward snapshot events from a public webhook endpoint

Forward thin events from a public webho

Use the following command to forward snapshot events from a public webhook endpoint to your local listener.

Command Line



```
$ stripe listen --load-from-webhooks-api --forward-to localhost:4242/webhook
```

To check webhook signatures, use the `{{WEBHOOK_SIGNING_SECRET}}` from the initial output of the listen command.



```
1 Ready! Your webhook signing secret is '{{WEBHOOK_SIGNING_SECRET}}' (^C to quit)
```

Triggering test events

To send test events, trigger an event type that your event destination is subscribed to by manually creating an object in the Stripe Dashboard. Learn how to trigger events with [Stripe for VS Code](#).

[Trigger a snapshot event](#)[Trigger a thin event](#)

You can use the following command in either [Stripe Shell](#) or [Stripe CLI](#). This example triggers a `payment_intent.succeeded` event:

Command Line



```
$ stripe trigger payment_intent.succeeded
> Running fixture for: payment_intent
> Trigger succeeded! Check dashboard for event details.
```

3 Register your endpoint



After testing your webhook endpoint function, use the [API](#) or the **Webhooks** tab in Workbench to register your webhook endpoint's accessible URL so Stripe knows where to deliver events. You can register up to 16 webhook endpoints with Stripe. Registered webhook endpoints must be publicly accessible **HTTPS** URLs.

Webhook URL format

The URL format to register a webhook endpoint is:



```
1 https://<your-website>/<your-webhook-endpoint>
```

For example, if your domain is `https://mycompanysite.com` and the route to your webhook endpoint is `@app.route('/stripe_webhooks', methods=['POST'])`, specify `https://mycompanysite.com/stripe_webhooks` as the **Endpoint URL**.

Create an event destination for your webhook endpoint

Create an event destination using Workbench in the Dashboard or programmatically with the [API](#) . You can register up to 16 event destinations on each Stripe account.

[Dashboard](#)[API](#)

To create a new webhook endpoint in the Dashboard:

- 1 Open the [Webhooks](#) tab in Workbench.
- 2 Click **Create an event destination**.
- 3 Select where you want to receive events from. Stripe supports two types of configurations: **Your account** and **Connected accounts**. Select **Account** to listen to events from your own account. If you created a [Connect application](#) and want to listen to events from your connected accounts, select **Connected accounts**.
- 4 Select the API version for the [events object](#) you want to consume.
- 5 Select the [event types](#) that you want to send to a webhook endpoint.
- 6 Select **Continue**, then select **Webhook endpoint** as the destination type.
- 7 Click **Continue**, then provide the **Endpoint URL** and an optional description for the webhook.

Listen to events from an organization webhook endpoint

If you create a webhook endpoint in an [organization account](#), select **Accounts** to listen to events from accounts in your organization. If you have [Connect platforms](#) as members of your organizations and want to listen to events from the all the platforms' connected accounts, select **Connected accounts**.

Note

[Workbench](#) replaces the existing [Developers Dashboard](#). If you're still using the Developers Dashboard, see how to [create a new webhook endpoint](#).

4 Secure your endpoint



After confirming that your endpoint works as expected, secure it by implementing [webhook best practices](#).

Secure your integration by making sure your handler verifies that all webhook requests are generated by Stripe. You can verify webhook signatures using our official libraries or verify them manually.

[Verify with official libraries \(recommended\)](#)[Verify manually](#)

Verify webhook signatures with official libraries

We recommend using our official libraries to verify signatures. You perform the verification by providing the event payload, the `Stripe-Signature` header, and the endpoint's secret. If verification fails, you get an error.



If you get a signature verification error, read our guide about [troubleshooting it](#).

Warning

Stripe requires the raw body of the request to perform signature verification. If you're using a framework, make sure it doesn't manipulate the raw body. Any manipulation to the raw body of the request causes the verification to fail.

Ruby



```
1 # Don't put any keys in code. See https://docs.stripe.com/keys-best-practices.
2 # Find your keys at https://dashboard.stripe.com/apikeys.
3 Stripe.api_key = 'sk_test_51TEzanK737GX6a3bG2...Obxr7Pr3V00iqbArLe9'
4
5 require 'stripe'
6 require 'sinatra'
7
8 # If you are testing your webhook locally with the Stripe CLI you
9 # can find the endpoint's secret by running `stripe listen`
10 # Otherwise, find your endpoint's secret in your webhook settings in
11 # the Developer Dashboard
12 endpoint_secret = 'whsec_...'
13
14 # Using the Sinatra framework
15 set :port, 4242
16
17 post '/my/webhook/url' do
18   payload = request.body.read
19   if header['HTTP_STRIPE_SIGNATURE']
```



Debug webhook integrations

Multiple types of issues can occur when delivering events to your webhook endpoint:

Stripe might not be able to deliver an event to your webhook endpoint.

Your webhook endpoint might have an SSL issue.

Your network connectivity is intermittent.

Your webhook endpoint isn't receiving events that you expect to receive.

View event deliveries

To view event deliveries, select the webhook endpoint under **Webhooks**, then select the **Events** tab. The **Events** tab provides a list of events and whether they're **Delivered**, **Pending**, or **Failed**. Click an event to view metadata, including the HTTP status code of the delivery attempt and the time of pending future deliveries.

You can also use the **Stripe CLI** to **listen for events** directly in your terminal.

Fix HTTP status codes

When an event displays a status code of `200`, it indicates successful delivery to the webhook endpoint. You might also receive a status code other than `200`. View the table below for a list of common HTTP status codes and recommended solutions.



PENDING WEBHOOK STATUS	DESCRIPTION	FIX
(Unable to connect) ERR	We're unable to establish a connection to the destination server.	Make sure that your host domain is publicly accessible to the internet.
(<code>302</code>) ERR (or other <code>3xx</code> status)	The destination server attempted to redirect the request to another location. We consider redirect responses to webhook requests as failures.	Set the webhook endpoint destination to the URL resolved by the redirect.
(<code>400</code>) ERR (or other <code>4xx</code> status)	The destination server can't or won't process the request. This might occur when the server detects an error (<code>400</code>), when the destination URL has access restrictions, (<code>401</code> , <code>403</code>), or when the destination URL doesn't exist (<code>404</code>).	Make sure that your endpoint is publicly accessible to the internet. Make sure that your endpoint accepts a POST HTTP method.
(<code>500</code>) ERR (or other <code>5xx</code> status)	The destination server encountered an error while processing the request.	Review your application's logs to understand why it's returning a <code>500</code> error.
(TLS error) ERR	We couldn't establish a secure connection to the destination server. Issues with the SSL/TLS certificate or an intermediate certificate in the destination server's certificate chain usually cause these errors. Stripe requires TLS version <code>v1.2</code> or higher.	Perform an SSL server test to find issues that might cause this error.
(Timed out) ERR	The destination server took too long to respond to the webhook request.	Make sure you defer complex logic and return a successful response immediately in your webhook handling code.

Event delivery behaviors

This section helps you understand different behaviors to expect regarding how Stripe sends events to your webhook endpoint.

Automatic retries



Stripe attempts to deliver events to your destination for up to three days with an exponential back off in live mode. View when the next retry will occur, if applicable, in your event destination's **Event deliveries** tab. We retry event deliveries created in a sandbox three times over the course of a few hours. If your destination has been disabled or deleted when we attempt a retry, we prevent future retries of that event. However, if you disable and then re-enable the event destination before we're able to retry, you still see future retry attempts.

Manual retries

There are two ways to manually retry events:

In the Stripe Dashboard, click **Resend** when looking at a specific event. This works for up to 15 days after the event creation.

With the **Stripe CLI**, run the `stripe events resend <event_id> --webhook-endpoint=<endpoint_id>` command. This works for up to 30 days after the event creation.

Manually resending an event that had previous delivery failures to a webhook endpoint doesn't dismiss Stripe's **automatic retry behavior**, even if it results in a `2xx` status code. Learn how to [process undelivered webhook events](#) to stop future retries.

Event ordering

Stripe doesn't guarantee the delivery of events in the order that they're generated. For example, creating a subscription might generate the following events:

`customer.subscription.created`

`invoice.created`

`invoice.paid`

`charge.created` (if there's a charge)

Make sure that your event destination isn't dependent on receiving events in a specific order. Be prepared to manage their delivery appropriately. You can also use the API to retrieve any missing

objects. For example, you can retrieve the invoice, charge, and subscription objects with the information from `invoice.paid` if you receive this event first.

API versioning



The API version in your account settings when the event occurs dictates the API version, and therefore the structure of an **Event** sent to your destination. For example, if your account is set to an older API version, such as 2015-02-16, and you change the API version for a specific request with **versioning**, the **Event** object generated and sent to your destination is still based on the 2015-02-16 API version. You can't change **Event** objects after creation. For example, if you update a charge, the original charge event remains unchanged. As a result, subsequent updates to your account's API version don't retroactively alter existing **Event** objects. Retrieving an older **Event** by calling `/v1/events` using a newer API version also has no impact on the structure of the received event. You can set test event destinations to either your default API version or the latest API version. The **Event** sent to the destination is structured for the event destination's specified version.

Best practices for using webhooks

Review these best practices to make sure your webhook endpoints remain secure and function well with your integration.

Handle duplicate events

Webhook endpoints might occasionally receive the same event more than once. You can guard against duplicated event receipts by logging the **event IDs** you've processed, and then not processing already-logged events.

In some cases, two separate Event objects are generated and sent. To identify these duplicates, use the ID of the object in `data.object` along with the `event.type`.

Only listen to event types your integration requires

Configure your webhook endpoints to receive only the types of events required by your integration. Listening for extra events (or all events) puts undue strain on your server and we don't recommend it.

You can **change the events** that a webhook endpoint receives in the Dashboard or with the API.

Handle events asynchronously

Configure your handler to process incoming events with an asynchronous queue. You might encounter scalability issues if you choose to process events synchronously. Any large spike in webhook deliveries (for example, during the beginning of the month when all subscriptions renew) might overwhelm your endpoint hosts.



Asynchronous queues allow you to process the concurrent events at a rate your system can support.

Exempt webhook route from CSRF protection

If you're using Rails, Django, or another web framework, your site might automatically check that every POST request contains a *CSRF token*. This is an important security feature that helps protect you and your users from **cross-site request forgery** attempts. However, this security measure might also prevent your site from processing legitimate events. If so, you might need to exempt the webhooks route from CSRF protection.

Rails



```
1 class StripeController < ApplicationController
2   # If your controller accepts requests other than Stripe webhooks,
3   # you'll probably want to use `protect_from_forgery` to add CSRF
4   # protection for your application. But don't forget to exempt
5   # your webhook route!
6   protect_from_forgery except: :webhook
7
8   def webhook
9     # Process webhook data in `params`
10  end
11 end
```

Receive events with an HTTPS server

If you use an HTTPS URL for your webhook endpoint (required in live mode), Stripe validates that the connection to your server is secure before sending your webhook data. For this to work, your server must be correctly configured to support HTTPS with a valid server certificate. Stripe webhooks support only **TLS** versions v1.2 and v1.3.

Roll endpoint signing secrets periodically

The secret used for verifying that events come from Stripe is modifiable in the **Webhooks** tab in Workbench. To keep them safe, we recommend that you roll (change) secrets periodically, or when you suspect a compromised secret.



To roll a secret:

- 1 Click each endpoint in the Workbench **Webhooks** tab that you want to roll the secret for.
- 2 Navigate to the overflow menu (***) and click **Roll secret**. You can choose to immediately expire the current secret or delay its expiration for up to 24 hours to allow yourself time to update the verification code on your server. During this time, multiple secrets are active for the endpoint. Stripe generates one signature per secret until expiration.

Verify events are sent from Stripe

Without verification, an attacker could send fake webhook events to your endpoint to trigger actions like fulfilling orders, granting account access, or modifying records. Always verify that webhook events originate from Stripe before acting on them.

Use both of these protections:

IP allowlisting: Stripe sends webhook events from a set list of **IP addresses**. Configure your server or firewall to only accept webhook requests from these addresses.

Signature verification: Stripe signs every webhook event by including a signature in the `Stripe-Signature` header. Verify this signature using our **official libraries** or **manually** to confirm the event wasn't sent or modified by a third party.

The following section describes how to verify webhook signatures:

- 1 Retrieve your endpoint's secret.
- 2 Verify the signature.

Retrieving your endpoint's secret

Use Workbench and go to the **Webhooks** tab to view all your endpoints. Select an endpoint that you want to obtain the secret for, then click **Click to reveal**.

Stripe generates a unique secret key for each endpoint. If you use the same endpoint for both **test and live API keys**, the secret is different for each one. Additionally, if you use multiple endpoints, you must obtain a secret for each one you want to verify signatures on. After this setup, Stripe starts to sign each webhook it sends to the endpoint.

Preventing replay attacks

A **replay attack** is when an attacker intercepts a valid payload and its signature, then re-transmits them. To mitigate such attacks, Stripe includes a timestamp in the `Stripe-Signature` header. Because this timestamp is part of the signed payload, it's also verified by the signature, so an attacker can't change the timestamp without invalidating the signature. If the signature is valid but the timestamp is too old, you can have your application reject the payload.

Our libraries have a default tolerance of 5 minutes between the timestamp and the current time. You can change this tolerance by providing an additional parameter when verifying signatures. Use Network Time Protocol (**NTP**) to make sure that your server's clock is accurate and synchronizes with the time on Stripe's servers.

Common mistake

Don't use a tolerance value of `0`. Using a tolerance value of `0` disables the recency check entirely.

Stripe generates the timestamp and signature each time we send an event to your endpoint. If Stripe retries an event (for example, your endpoint previously replied with a non-`2xx` status code), then we generate a new signature and timestamp for the new delivery attempt.

Quickly return a 2xx response

Your **endpoint** must quickly return a successful status code (`2xx`) prior to any complex logic that could cause a timeout. For example, you must return a `200` response before updating a customer's invoice as paid in your accounting system.

See also

-  [Send events to Amazon EventBridge](#)
-  [List of thin event types](#)
-  [List of snapshot event types](#)
-  [Interactive webhook endpoint builder](#)

Was this page helpful?

 Yes

 No

 Need help? [Contact Support](#).

 Chat with Stripe developers on [Discord](#).

 Check out our [changelog](#).

 Questions? [Contact Sales](#).

 LLM? [Read llms.txt](#).



Powered by [Markdoc](#)

